DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Structured Streaming

Section 1. Triggers, checkpoints, watermarks, and what you may change on restart.

SECTION 1 of 10 WEIGHT 22% TOPIC Streaming

admin@sthithapragna.com

The checkpoint **is** the query. Change the query enough and the checkpoint no longer describes it.

What is actually stored

CONTENTS	WHY IT MATTERS
Source offsets	How far into each source the query has read. This is what makes restart incremental.
Intermediate state	Accumulated data for aggregations, joins and deduplication.
Metadata	Configuration and a unique identity for the query.
Commit log	Which micro-batches completed. This is what prevents reprocessing.

ON THE EXAM

Every query needs its own checkpoint directory in durable cloud storage. Two queries sharing one location is a corruption waiting to happen.

The checkpoint you did not specify

- Some sinks **generate a temporary checkpoint automatically** if you omit the location. Display output and the memory sink both do.
- Those temporary locations offer **no fault tolerance and no consistency guarantee**, and may not be cleaned up.
- Databricks recommends **always specifying a checkpoint location**, even for these.
- It is the difference between something that appears to work in a notebook and something that survives a restart.

> ON THE EXAM
>
> A stream that works interactively and behaves oddly in production is often a stream that was never given a real checkpoint.

How often the query looks for work

TRIGGER	BEHAVIOUR
Default	Starts the next micro-batch as soon as the previous one finishes.
processingTime interval	Fixed interval micro-batches. Checks for new data every N seconds.
processingTime of 0	Next batch begins immediately when data is available. Lowest micro-batch latency.
availableNow	Consumes all available records as an incremental batch, then stops.
Once	Deprecated from DBR 11.3 LTS. Use availableNow instead.
Continuous	Not supported on Databricks. Experimental in open-source Spark only.

ON THE EXAM

On serverless compute only availableNow and Once are supported, and Databricks recommends availableNow.

Incremental batch against operational streaming

AVAILABLENOW

Process what is there, then stop

- Compute runs only while there is work
- Right for periodic incremental loads
- Batch size tunable with rate limits
- The recommended trigger on serverless

PROCESSINGTIME OF 0

Keep going, as fast as possible

- Lowest micro-batch latency
- Right for fraud, alerting, monitoring
- Costs more in cloud storage API calls
- Explicitly not availableNow or Once

STATED IN THE DOCS

For operational workloads the docs say plainly: do not use availableNow, Once or Continuous. Those are for incremental batch, not for low latency.

Real-time mode

- Achieves end-to-end latency **under one second at the tail**, and around **300 milliseconds** typically.
- This is what replaced the idea of continuous processing, which was never supported here.
- For operational latency targets in the tens to hundreds of milliseconds, this is the answer.
- Micro-batch tuning cannot reach these numbers. There is a floor to how short a batch can usefully be.

> ON THE EXAM
>
> If a question sets a sub-second latency requirement, micro-batch tuning is not the answer being looked for.

What gets written each trigger

MODE	WHAT IT EMITS, AND THE CATCH
Append	Only new rows, never revised. Required for stream-stream joins.
Update	Rows that changed, re-emitted until the watermark expires. The sink must be idempotent.
Complete	The entire result table every trigger. Not for low latency.

ON THE EXAM

Update mode gives the lowest latency where it is supported, but it re-emits the same key repeatedly. A sink that cannot absorb that will duplicate rows.

Multiple stateful operators

- From **DBR 13.3 LTS** you can chain stateful operators, feeding a windowed aggregation into a join.
- **Only append output mode is supported** when you do.
- FlatMapGroupsWithState is not supported in a chain.
- So chaining and update mode are mutually exclusive. Choose which property you need.

> ON THE EXAM
>
> A question that chains stateful operations and then asks for update mode is describing something that does not work.

The trade-off you are actually setting

- A watermark says how long the query keeps **accepting out-of-order event-time data** before dropping it and evicting state.
- Too short and it **silently discards valid late records**. Nothing errors.
- Too long and you carry more state, and pay in latency and memory.
- A small multiple of your latency target, such as **twice** it, is a reasonable starting point.

> ON THE EXAM
>
> The failure mode is silence. A watermark that is too short does not fail loudly; it just quietly loses data you needed.

What you may change and still resume

CHANGE	CAN YOU KEEP THE CHECKPOINT?
Adding or removing a filter	Yes. Generally safe.
Rate limits, such as max files per trigger	Yes.
Trigger interval	Yes. Can be changed between runs.
Number or type of input sources	No. Needs a new checkpoint.
Kafka topics or Auto Loader paths	No.
Stateful operator type, or state schema	No.
Output sink type	No.

ON THE EXAM

The pattern: anything that changes what state means, or which sources exist, breaks the checkpoint. Anything that changes only pace or filtering does not.

Sources are identified by position

- By default Structured Streaming identifies sources by their **position in the query plan**, not by name.
- So **reordering** sources breaks checkpoint compatibility, even though nothing about them changed.
- **Source evolution**, from DBR 18.2, lets you name each source and then reorder, add or remove them.
- Deduplication operators can also fail to restart when the compute access mode changes.

> ON THE EXAM
>
> Adding a source to an existing union is the classic version of this. It looks additive and it invalidates the checkpoint.

The state store and how to make it cheaper

- The **RocksDB state store** is required for both changelog checkpointing and asynchronous state checkpointing.
- **Changelog checkpointing** persists only the incremental changes to state, rather than the whole thing.
- **Asynchronous state checkpointing** overlaps the checkpoint write with the next micro-batch.
- The state management scheme **cannot be changed** between restarts.

> ON THE EXAM
>
> Reach for these when checkpoint writing is the bottleneck in your batch duration, not before. Async checkpointing has recovery and resizing caveats.

Asynchronous progress tracking

- Updates the offset and commit logs **without blocking data processing**.
- Useful when offset checkpointing is adding latency to a continuously running stream.
- **Not compatible with the availableNow or Once triggers.**
- So it belongs with operational streaming, alongside a processing time trigger.

> ON THE EXAM
>
> Two separate optimisations with similar names: state checkpointing is about accumulated state, progress tracking is about offsets.

The rules Databricks states directly

- Run streams as **Lakeflow Jobs on jobs compute**. Never on all-purpose compute.
- Schedule with the job's **Continuous mode**. That is job scheduling, not the streaming trigger.
- **Do not enable autoscaling** for Structured Streaming jobs. Scaling down has known limitations.
- Remove display and count calls from production notebooks.

> ON THE EXAM
>
> Continuous job mode prevents concurrent runs, restarts on failure, and uses exponential backoff. That is the restart-on-failure requirement, solved.

Where exactly-once stops

- **foreachBatch provides at-least-once**, not exactly-once. This is stated explicitly.
- So anything written through it must be **idempotent**, or duplicates will appear after a retry.
- Checkpoints plus an idempotent sink are what deliver exactly-once for the sources that support it.
- Update output mode carries the same requirement, for the same reason.

> ON THE EXAM
>
> Exactly-once is a property of the whole path, not of Structured Streaming alone. The sink has to hold up its end.

Multiple tasks or multiple queries

	RETRY BEHAVIOUR
Multiple tasks in one job	All tasks must fail before the job retries.
Multiple queries in one task	The task retries if any single query fails.

ON THE EXAM

With multiple tasks, compute is sized per task. With multiple queries, they share compute and can be assigned to scheduler pools.

Do not await termination inside a job

- In Lakeflow Jobs, **do not call awaitTermination**. The Jobs service already prevents a run finishing while a query is active.
- It blocks the cell, stops the service tracking the query, and **breaks backlog metrics and notifications**.
- Use it in interactive notebooks, so query state is visible and failures surface.
- And locally, where the process would otherwise exit as soon as the main thread finished.

> ON THE EXAM
>
> The same line of code is correct in a notebook and wrong in a job. Context decides.

What to carry into the exam

- The checkpoint records offsets, state, metadata and commits. Change what state **means** and it cannot be reused.
- Once is **deprecated**, Continuous is **unsupported**. availableNow for incremental batch, processingTime of 0 for operational.
- A watermark that is too short **silently drops** valid data.
- Update mode and foreachBatch both require an **idempotent sink**.
- Jobs compute, continuous job scheduling, no autoscaling.

> ON THE EXAM
>
> When a restart fails after a code change, ask what the change did to the state or the sources. That is where the answer nearly always is.

A streaming query aggregates orders by region and writes to Delta. To improve reporting granularity, a developer changes the grouping from region alone to region and product category, and redeploys.

The query fails on restart with an error about the checkpoint. The developer points out that they only changed a group by clause, and no sources or sinks were touched.

Why does the restart fail?

A The output sink schema changed, and Delta requires schema evolution to be enabled first.

B Changing the grouping keys changes the state schema, which the checkpoint cannot carry forward.

C Streaming aggregations cannot be modified at all once a query has been started.

D The trigger interval must be reset whenever the query plan changes.

◆ PAUSE AND DECIDE

Why does the restart fail?

A The output sink schema changed, and Delta requires schema evolution to be enabled first.

Plausible-sounding, but the failure is about state, not the sink's table schema.

B Changing the grouping keys changes the state schema, which the checkpoint cannot carry forward.

State schema is on the not-allowed list. Any change to the number or type of grouping keys invalidates it.

C Streaming aggregations cannot be modified at all once a query has been started.

Too absolute. Filters and rate limits can change freely; it is the state schema that cannot.

D The trigger interval must be reset whenever the query plan changes.

Trigger interval is explicitly safe to change on the same checkpoint.

ON THE EXAM

The developer's reasoning is the trap. A small code change can still be a large state change, and the checkpoint tracks state rather than lines of code.

A fraud detection stream must act on card transactions within a few hundred milliseconds. The current implementation uses a 10-second processingTime trigger and writes with complete output mode.

The team has tried reducing the trigger to 1 second and then 500 milliseconds. Latency improved but plateaued well above the target, and cost rose sharply.

What is the appropriate change?

A Reduce the trigger interval further, to 100 milliseconds.

B Switch to the availableNow trigger, which removes the fixed interval entirely.

C Move to real-time mode, and change the output mode away from complete.

D Enable asynchronous state checkpointing to remove the checkpoint write from the batch.

◆ PAUSE AND DECIDE

What is the appropriate change?

A Reduce the trigger interval further, to 100 milliseconds.

The plateau is the point. Micro-batch overhead is a floor no interval gets under.

B Switch to the availableNow trigger, which removes the fixed interval entirely.

availableNow stops after processing. The docs say explicitly not to use it for operational workloads.

C Move to real-time mode, and change the output mode away from complete.

Real-time mode reaches around 300 ms, and complete mode is explicitly not for low latency.

D Enable asynchronous state checkpointing to remove the checkpoint write from the batch.

Helps only when checkpoint writing is the bottleneck. Nothing here says it is.

ON THE EXAM

The word plateaued is the clue. Diminishing returns from tuning means you have hit a structural limit, not a configuration one.

A stream unions events from two Kafka topics and writes to a bronze streaming table. A third topic must now be added to the same query.

The team adds it to the union, redeploys, and the query fails on restart. They confirm the two original topics are unchanged and the sink is untouched.

What explains this, and what is the modern remedy?

A Adding a source changes the number of input sources, which breaks the checkpoint. Source evolution allows named sources to be added.

B Kafka topics must be subscribed with a pattern rather than a list, so no new source is created.

C The union operation is stateful, so any change to it requires clearing the state store.

D Each Kafka topic requires its own checkpoint, so the query needs three checkpoint locations.

◆ PAUSE AND DECIDE

What explains this, and what is the modern remedy?

A Adding a source changes the number of input sources, which breaks the checkpoint. Source evolution allows named sources to be added.

Sources are identified by position by default. Source evolution assigns stable names so they can be added and reordered.

B Kafka topics must be subscribed with a pattern rather than a list, so no new source is created.

Changing the subscribed topics is itself on the not-allowed list.

C The union operation is stateful, so any change to it requires clearing the state store.

A union is not a stateful operator. The problem is source identity, not state.

D Each Kafka topic requires its own checkpoint, so the query needs three checkpoint locations.

One checkpoint per query, not per source.

ON THE EXAM

Adding a source feels purely additive and is not. Position-based identity is why, and named sources are the fix.

A stream uses foreachBatch to write aggregated results into an external operational database via a JDBC connection. The job is configured to restart automatically on failure.

After an executor failure and automatic restart, the operations team reports duplicate rows in the target database for the period around the failure.

What is the correct explanation?

A The checkpoint was corrupted by the executor failure and replayed committed offsets.

B foreachBatch provides at-least-once, so the write must be made idempotent.

C Automatic restart should be disabled, since it causes reprocessing.

D The output mode should be complete so the target is fully rewritten each batch.

◆ PAUSE AND DECIDE

What is the correct explanation?

A The checkpoint was corrupted by the executor failure and replayed committed offsets.

Nothing suggests corruption. This behaviour is the documented guarantee, not a fault.

B foreachBatch provides at-least-once, so the write must be made idempotent.

Stated explicitly in the docs. A batch can be re-executed after failure, so the write must tolerate it.

C Automatic restart should be disabled, since it causes reprocessing.

Databricks recommends automatic restart. Removing it trades duplicates for an outage.

D The output mode should be complete so the target is fully rewritten each batch.

Rewriting an operational database every batch is not viable, and does not address the guarantee.

ON THE EXAM

Exactly-once is a property of the whole path. Writing through an arbitrary sink hands the guarantee to you.

A sessionisation stream uses a 30-second watermark on event time. Analysts report that a small but material number of mobile events are missing from the results entirely. Mobile clients buffer events when offline and can upload them up to 20 minutes later.

No errors appear in the logs, and the stream reports healthy throughput.

What is happening, and what is the trade-off in fixing it?

A The watermark is dropping late events silently. Lengthening it retains them, at the cost of more state and latency.

B The trigger interval is too long, so events arrive after the batch closes. Shorten the interval.

C Append output mode is discarding revised rows. Switch to update mode.

D The state store has evicted entries under memory pressure. Enable RocksDB.

◆ PAUSE AND DECIDE

What is happening, and what is the trade-off in fixing it?

A The watermark is dropping late events silently. Lengthening it retains them, at the cost of more state and latency.

A 30-second watermark against 20-minute lateness. The watermark drops what arrives beyond it, with no error.

B The trigger interval is too long, so events arrive after the batch closes. Shorten the interval.

Trigger interval governs when batches run, not how much lateness is tolerated.

C Append output mode is discarding revised rows. Switch to update mode.

These events never enter the result at all. Output mode governs what is emitted, not what is accepted.

D The state store has evicted entries under memory pressure. Enable RocksDB.

Eviction here is by watermark policy, not memory pressure.

ON THE EXAM

Silence is the signature. A watermark that is too short produces no error, no warning, and no entry in the logs.

A team runs eight streaming queries. They are currently defined as eight queries inside a single task, on one autoscaling all-purpose cluster, and the notebook calls awaitTermination at the end.

One query fails roughly weekly, taking all eight down. They want a more resilient arrangement.

Which set of changes best addresses this?

A Keep one task, add scheduler pools, and increase the autoscaling maximum.

B Split into separate tasks on jobs compute, disable autoscaling, use continuous job scheduling, and remove awaitTermination.

C Keep the single task but wrap each query in retry logic inside the notebook.

D Move all eight queries onto a single availableNow trigger so failures stop the batch cleanly.

◆ PAUSE AND DECIDE

Which set of changes best addresses this?

A Keep one task, add scheduler pools, and increase the autoscaling maximum.

Scheduler pools help contention, not isolation. Autoscaling is advised against for streaming.

B Split into separate tasks on jobs compute, disable autoscaling, use continuous job scheduling, and remove awaitTermination.

Separate tasks isolate the failure, and every other element is a stated Databricks recommendation.

C Keep the single task but wrap each query in retry logic inside the notebook.

Rebuilds by hand what continuous job scheduling already provides, and keeps every other problem.

D Move all eight queries onto a single availableNow trigger so failures stop the batch cleanly.

Converts continuous streams into periodic batches, changing the workload rather than fixing it.

ON THE EXAM

Four separate recommendations are being broken at once here. The question rewards recognising the whole set rather than the loudest one.